Recommendation System Project

MovieLens 100k

Words: 1425

Advanced Databases

Professora Márcia Barros

December 2025

André Franco
fc59876@alunos.fc.ul.pt

Joana Pimentel Dias de Deus
fc58197@alunos.fc.ul.pt

Tiago Filipe de Deus Folgado Pereira
fc55854@alunos.fc.ul.pt

Tiago Matos Frade
fc59795@alunos.fc.ul.pt

Index

1. Project Setup 2
2. Goal 1 – Data Selection, Modeling, and Database Creation 2
   - 2.1. Data Selection 2
   - 2.2. Schema 2
3. Goal 2 – Query Design and Implementation 3
   - 3.1. Query Design 3
   - 3.2. Movie Recommendation 3
4. Goal 3 – Concurrency Testing 3
5. Goal 4 – Performance Optimization and Integration Tuning 4
   - 5.1. Schema Optimization 4
   - 5.2. DB Updates 5
   - 5.3. Query Optimization 5
   - 5.4. Indexing 6
6. Appendix 7

Project Setup

To see the work, first run the file *Schemas_after_Tuning.ipynb*: it creates the databases, populates them and creates the indexes. It might take up to five minutes due to the use of API's (MongoDB). After that, the other files can be ran at any point, by any order:

1. *queries.ipynb*;
2. *Concurrency.ipynb*;
3. *rebuild.ipynb*: this program serves to maintain consistency between MySQL and MongoDB. For example, if a user makes a rating, recalculations must be done, and it's this program that does it.

Goal 1 – Data Selection, Modeling, and Database Creation

Data Selection

For the development of this project we decided to use the MovieLens 100k dataset. This dataset consists of 100 000 ratings, from 1 to 5, from 943 users on 1682 movies. Each user rated at least 20 movies, and the data was collected through the MovieLens website during a seven month period from September 19th 1997 to April 22nd 1998.

Data clean-up was done by the authors, removing users and their respective ratings when they had rated less than 20 movies.

In addition to the data available on the dataset, we added three parameters to the users' information: location, latitude and longitude. These were calculated through API's, using the zip code available as a reference. We used this information to create geospatial indexes for goal 4.

Schema

Initially we had decided to add static data to MySQL, data that didn't require change as often, such as a user's profile or a movie's information. This was the intended objective for MySQL. For MongoDB, however, we didn't quite grasp what was needed, as the professor was expecting.

We initially understood that MongoDB was supposed to work just like the MySQL database. When testing queries, concurrency and indexing, since they had the same information and structure (to a certain extent), it would perfectly showcase their differences

(SQL vs NOSQL).

It was later that we realized that MongoDB was supposed to function as a support to MySQL, where calculus, and function results go to, the new format reflects this, and works as a support to MySQL, leaving MySQL very light on the setup and data integration, the exact opposite as MongoDB. See appendix A for more details about the initial schema.

Goal 2 – Query Design and Implementation

Query Design

For the queries we thought about some questions that a user could be asking:

- What are the movies people like me have watched?
- What are the best movies from the year 1996?
- What are some movies from the genre Animation?
- What are the most recent ratings?

We created two simple queries, two complex queries, an INSERT, UPDATE and DELETE. See appendix B for more details about the initial queries.

Movie Recommendation

After feedback from the teacher, we decided to create a function ***recommend(user_id)***. This function recommends movies based on a user's favourite genre, using the genre relations (established by us) to recommend movies, not yet seen by the user, of genres related. The number of movies displayed can be changed by changing the parameter *"limit"* (by default *limit=1*).

Goal 3 – Concurrency Testing

The purpose of this goal was to simulate concurrency (multiple concurrent users accessing and updating both databases) measuring the response time, conflicts or deadlocks and consistency of cross-database operations.

The principal conclusions that were drawn from this goal, following the given environment configuration (MySQL: 9.4, Client: MySQL Command Line Client 9.4, Base: movielens_relational, MongoDB: 7.0.6, Client: MongoDB Compass 1.48.2, Base: movielens_modular, Language: Python 3.11, Libraries: mysql-connector-python, pymongo,

threading, time, tabulate, pandas), strategy test (Simulated concurrency, consistency control, measurement) and test flux (initiates in MongoDB by changing ratings, then MySQL, after that, it reads final ratings, verifies cross-database consistency, and reverts ratings to initial ones to maintain consistency for future tests) was:

- The tests showed that employing row-level locking in MySQL, via SELECT.. FOR UPDATE, avoided writing conflicts without generating deadlocks in the given scenario, it did, however, come at the cost of latency when multiple threads competed for the same line.
- In MongoDB.The atomic operations of find_one_and_update assure consistent updates per document.
- Concluding, the reinitialization/restore assured cross-database consistency after the test, confirming that the system correctly handles concurrency within the defined context.

  All of this is done in *Concurrency.ipynb*.

Goal 4 – Performance Optimization and Integration Tuning

Schema Optimization

To better fit the requirements and after receiving feedback about our original schema, we tuned it. MySQL didn't suffer many modifications. From goal 1 until now we had added the three new parameters to the user in both MySQL and MongoDB, but we ended up only having them in their own table MongoDB, as the locations would be appearing in repetition, and due to the use of API's, to calculate longitude and latitude from the zip code, as well as location, it would take longer to calculate everything not only for MongoDB, but also for MySQL.

For MongoDB we changed the data we stored there to be the type of data that would require frequent changes such as top ratings. Every time a user submits a new rating, the system needs to recalculate averages and, consequently, recalculate the top ratings. MongoDB is the best fit for this task because it has a flexible document structure as well as it can be updated relatively faster than MySQL. The changes we made are as it follows:

- Modified collection 'movies'. Now it only contains the id of the movie and the rating stats of the movie;
- Modified collection 'users', which stores users' favorite movies, favorite genres and rating stats (their average rating, how many ratings and their last ratings);
- Added collection 'location', which stores the top rated movies and genres from a location. It also stores the users that belong to that location, with their zip code, latitude and longitude;
- Added collections 'age', 'year', 'occupation', 'genre', 'gender' and 'ratings_total' which stores an ordered array of the top rated movies from each category.

  The code for these updates can be found in *queries.ipynb*.

DB Updates

After changing the MongoDB database, a need arose to update/rebuild the data when the MySQL data was changed. To satisfy such a requirement we created a set of functions to rebuild the different parts of the database, as well as a master function to update the whole thing, this gives us the flexibility to update everything or only the portions that actually need to be updated with the new data.

The code for this functionality is on the *rebuild.ipynb* file.

Query Optimization

Since there were new schemas, and after receiving feedback from what we presented on Goal 2, the queries had to be updated. We made them more dynamic, by randomly selecting, for example, a user on MySQL and then extracting relevant information from it, so it can then be used to extract top ratings from MongoDB. Each query answers dynamic questions such as "What are the X Y movies with rating W?", where X is either 'best' or 'worse', Y is the amount of movies to be displayed, and W is the rating available. Each of those values are randomly selected at the start of the query runs.

MySQL queries are mostly used in auxiliary functions to extract specific aspects, while MongoDB is used to retrieve the ratings associated with those aspects (e.g., age, occupation, etc.). With this change, the queries ran much faster, since before there were calculations happening while the query was running. Now, they cease to exist, and the queries simply retrieve information that was already previously calculated.

Code for the queries, with more information, can be found in the file *queries.ipynb*.

Indexing

To better understand the efficiency of different indexing strategies, we conducted four separate index experiments: a geospatial index, an index on *user_id*, an index for *movie_id*, and a composite index on *user_id_movie_id*. For each case, we executed the same query before and after creating the corresponding index and compared the time taken and the number of documents examined.

Overall, the results show that indexing has a significant impact on query performance. Both the geospatial index and the *movie_id* index produced the largest improvement, significantly reducing the number of documents examined (especially in the *movie_id* index) and decreasing execution time.

The *user_id* index was the only one with non-significant improvement. The number of matched documents remained the same and the performance, evaluated in time, did not improve quite that much.

In summary, our results prove that when properly chosen indexes can optimize query efficiency, particularly when filtering fields with high selectivity.

Appendix

A. Initial Schema

```sql
CREATE TABLE occupation (
    occupation_id INT PRIMARY KEY AUTO_INCREMENT,
    occupation VARCHAR(50)
);

CREATE TABLE users (
    user_id INT PRIMARY KEY,
    age INT,
    gender VARCHAR(10),
    occupation_id INT,
    zip_code VARCHAR(10),
    FOREIGN KEY (occupation_id) REFERENCES occupation(occupation_id)
);

CREATE TABLE movies (
    movie_id INT PRIMARY KEY,
    movie_title VARCHAR(200),
    release_date DATE NULL,
    imdb_url VARCHAR(200)
);

CREATE TABLE genre (
    genre_id INT PRIMARY KEY,
    genre VARCHAR(50)
);

CREATE TABLE genreofmovies (
    movie_id INT,
    genre_id INT,
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id),
    FOREIGN KEY (genre_id) REFERENCES genre(genre_id)
);

CREATE TABLE ratings (
    user_id INT,
    movie_id INT,
    rating INT,
    timestamp DATETIME,
    PRIMARY KEY (user_id, movie_id),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id)
);
```

Image 1 – Initial MySQL schema

```
Coleção 1 – users

Guarda dados demográficos e estatísticas resumidas de ratings.

db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["user_id", "age", "gender", "occupation", "zip_code"],
      properties: {
        _id: { bsonType: "objectId" },
        user_id: { bsonType: "int", description: "Unique user identifier" },
        age: { bsonType: "int", minimum: 1 },
        gender: { bsonType: "string", enum: ["M", "F"] },
        occupation: { bsonType: "string" },
        zip_code: { bsonType: "string" },
        rating_stats: {
          bsonType: "object",
          properties: {
            count: {
              bsonType: "int",
              description: "Total number of ratings given by the user"
            },
            avg_rating: {
              bsonType: "double",
              description: "Average rating given by the user"
            },
            last_rated_movies: {
              bsonType: "array",
              items: { bsonType: "int" },
              description: "IDs of the last movies rated"
            }
          }
        }
      }
    }
  }
});
```

Image 2 – Initial MongoDB schema

```
Coleção 2 – movies

Contém os metadados estáticos dos filmes (sem redundância).

db.createCollection("movies", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["movie_id", "movie_title", "release_date", "imdb_url", "genres"],
      properties: {
        _id: { bsonType: "objectId" },
        movie_id: { bsonType: "int" },
        movie_title: { bsonType: "string" },
        release_date: {
          bsonType: "string",
          pattern: "^\\d{4}-\\d{2}-\\d{2}$"
        },
        imdb_url: { bsonType: "string" },
        genres: {
          bsonType: "array",
          items: { bsonType: "string" }
        }
      }
    }
  }
});
```

Image 3 – Initial MongoDB schema

```
Coleção 3 — ratings

Armazena cada avaliação como um documento independente (escala perfeitamente).

db.createCollection("ratings", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["user_id", "movie_id", "rating", "timestamp"],
      properties: {
        _id: { bsonType: "objectId" },
        user_id: {
          bsonType: "int",
          description: "ID of the user who rated"
        },
        movie_id: {
          bsonType: "int",
          description: "ID of the movie rated"
        },
        rating: {
          bsonType: "int",
          minimum: 1,
          maximum: 5,
          description: "Rating value (1-5)"
        },
        timestamp: {
          bsonType: "int",
          description: "Unix timestamp of the rating"
        }
      }
    }
  }
});
```

Image 4 – Initial MongoDB schema

B. Queries for Goal 2

MySQL Queries

(Complex) Query q1 identifies the most highly rated movies among students (only counting ratings of 5).

Note: Tried this with MongoDB (see more down) and it takes longer, over 10 seconds.

```
q1 = """
SELECT
    m.movie_title,
    COUNT(*) AS num_5star_students
FROM ratings r
JOIN users u ON r.user_id = u.user_id
JOIN movies m ON r.movie_id = m.movie_id
WHERE
    r.rating = 5
    AND u.occupation_id = (
        SELECT occupation_id
        FROM occupation
        WHERE occupation = 'student'
    )
GROUP BY
    m.movie_title
ORDER BY
    num_5star_students DESC
LIMIT 10;
"""

with engine.connect() as conn:
    result = pd.read_sql(text(q1), conn)

print(result)
```

[22]

```
...                        movie_title  num_5star_students
0                 Star Wars (1977)                  76
1         Return of the Jedi (1983)                 54
2                   Titanic (1997)                  52
3   Empire Strikes Back, The (1980)                 51
4              Pulp Fiction (1994)                  47
5        Princess Bride, The (1987)                 46
6            Godfather, The (1972)                  44
7  Shawshank Redemption, The (1994)                 41
8    Raiders of the Lost Ark (1981)                 40
9                     Fargo (1996)                  39
```

Image 5 – MySQL queries for Goal 2

(Complex) Query q2 identifies the most highly rated movies (average rating) with release date of 1996.

Note: Tried this on MongoDB, but the time was over 20 seconds

```
q2 = """
SELECT
    m.movie_title,
    YEAR(m.release_date) AS release_year,
    ROUND(AVG(r.rating), 2) AS avg_rating,
    COUNT(r.rating) AS total_ratings
FROM movies m
JOIN ratings r ON m.movie_id = r.movie_id
WHERE YEAR(m.release_date) = 1996
GROUP BY m.movie_id, m.movie_title, release_year
HAVING COUNT(r.rating) >= 30
ORDER BY avg_rating DESC, total_ratings DESC
LIMIT 10;
"""

with engine.connect() as conn:
    result = pd.read_sql(text(q2), conn)

print(result)
```

```
                                     movie_title  release_year  \
0                             Close Shave, A (1995)          1996
1  Wallace & Gromit: The Best of Aardman Animatio...          1996
2                             Secrets & Lies (1996)          1996
3                                Sling Blade (1996)          1996
4                                 Braveheart (1995)          1996
5                                Taxi Driver (1976)          1996
6                                  Lone Star (1996)          1996
7                                      Shine (1996)          1996
8                                  Big Night (1996)          1996
9                              Fly Away Home (1996)          1996

   avg_rating  total_ratings
0        4.49            112
1        4.45             67
2        4.27            162
3        4.20            136
4        4.15            297
5        4.12            182
6        4.05            187
7        3.92            129
8        3.90            171
9        3.90             31
```

Image 6 – MySQL queries for Goal 2

(Insert) Inserting a new movie on the database.

```
q_insert ="""
INSERT INTO movies (movie_id, movie_title, release_date, imdb_url)
VALUES (1996, 'A Very Legit Movie', '1996-07-10', 'https://www.imdb.com/title/thistottallyexists/');
"""

with engine.connect() as conn:
    conn.execute(text(q_insert))
    conn.commit()

with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM movies WHERE movie_id = 1996;"))
    for row in result:
        print(row)
```

[24]

```
(1996, 'A Very Legit Movie', datetime.date(1996, 7, 10), 'https://www.imdb.com/title/thistottallyexists/')
```

(Update) Update the title of the movie we just inserted.

```
q_update = """
UPDATE movies
SET movie_title = 'Truly a Very Legit Movie'
WHERE movie_id = 1996;
"""

with engine.connect() as conn:
    conn.execute(text(q_update))
    conn.commit()

with engine.connect() as conn:
    result = conn.execute(text("SELECT * FROM movies WHERE movie_id = 1996;"))
    for row in result:
        print(row)
```

[25]

```
(1996, 'Truly a Very Legit Movie', datetime.date(1996, 7, 10), 'https://www.imdb.com/title/thistottallyexists/')
```

(Delete) Deleting a movie from the database.

```
q_delete = """
DELETE FROM movies
WHERE movie_id = 1996;
"""

with engine.connect() as conn:  # auto-commit on exit
    conn.execute(text(q_delete))
    conn.commit()
```

[26]

Image 7 – INSERT/UPDATE/DELETE queries for Goal 2

(Simple) Query to retrieve 10 movies from the genre Animation

```
for doc in db.movies.find({"genres": "Animation"}, {"_id": 0, "movie_title": 1}).limit(10):
    print(doc)
```

[28]

```
{'movie_title': 'Toy Story (1995)'}
{'movie_title': 'Lion King, The (1994)'}
{'movie_title': 'Aladdin (1992)'}
{'movie_title': 'Snow White and the Seven Dwarfs (1937)'}
{'movie_title': 'Heavy Metal (1981)'}
{'movie_title': 'Aristocats, The (1970)'}
{'movie_title': 'All Dogs Go to Heaven 2 (1996)'}
{'movie_title': 'Wallace & Gromit: The Best of Aardman Animation (1996)'}
{'movie_title': 'Wrong Trousers, The (1993)'}
{'movie_title': 'Grand Day Out, A (1992)'}
```

(Simple) Query to find the 10 most recent ratings

```
for doc in db.ratings.find({},{"_id": 0, "movie_id": 1, "rating": 1}).sort("timestamp", -1).limit(10):
    print(doc)
```

[29]

```
{'movie_id': 328, 'rating': 3}
{'movie_id': 313, 'rating': 3}
{'movie_id': 272, 'rating': 4}
{'movie_id': 689, 'rating': 4}
{'movie_id': 748, 'rating': 4}
{'movie_id': 300, 'rating': 4}
{'movie_id': 333, 'rating': 4}
{'movie_id': 322, 'rating': 4}
{'movie_id': 354, 'rating': 5}
{'movie_id': 362, 'rating': 4}
```

Image 8 – MongoDB queries for Goal 2